

① $0x25A1C \rightarrow \underbrace{\begin{matrix} 10 & 0101 & 1010 & 0001 & 1100 \\ 2 & 4 & 4 & 4 & 4 \end{matrix}}_{18 \text{ bits}}$

Gama de endereços = $0x00000$ a $0x3FFFF$

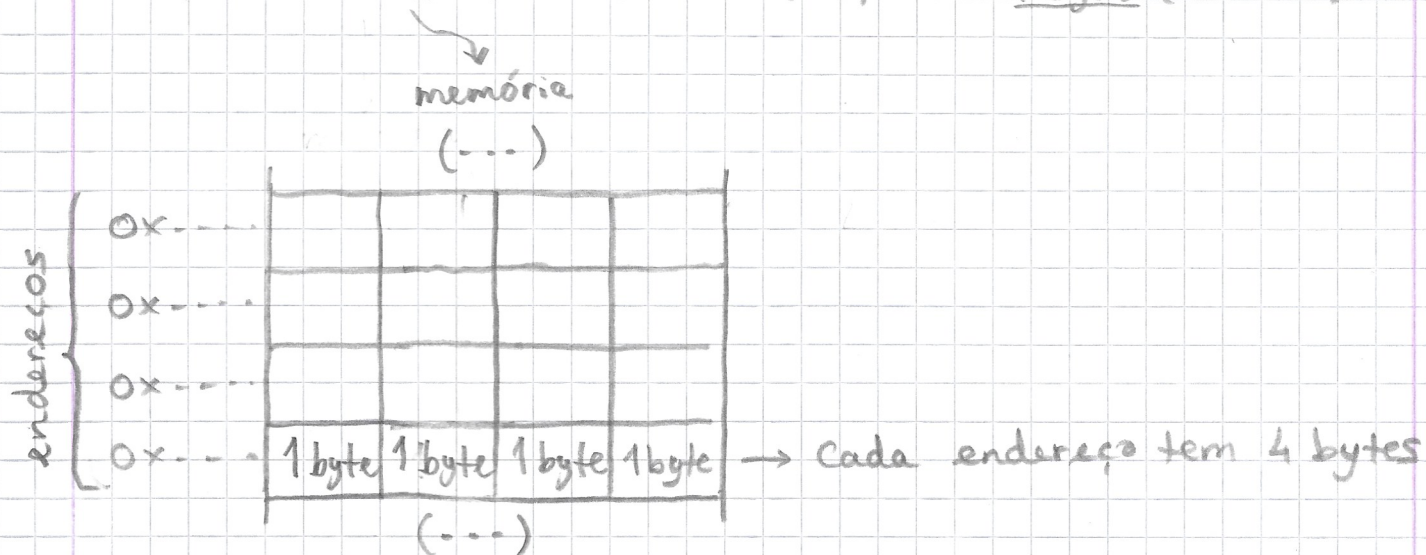
Logo, 18 bits $\rightarrow$ barramento de endereços

$2^{18} \rightarrow$ espaço de endereçamento

Resposta: b)

② byte-addressable $\rightarrow$ cada endereço tem 1 byte (8 bits)

word-addressable $\rightarrow$ cada endereço tem 4 bytes (32 bits)



Como o barramento de endereços é 24 bits



o espaço de endereçamento é 2^{24} bytes

$\rightarrow$ Se fosse byte-addressable a resposta era 2^{24} bytes

$\rightarrow$ Como é word-addressable, temos de fazer:

$$2^{24} \times 2^2 = \boxed{2^{26} \text{ bytes}}$$

4 (porque cada endereço tem 4 bytes)

Resposta: c)

③ single-cycle $\rightarrow$ 2 memórias $\rightarrow$ Harvard

operandos residem em registros internos ou na memória externa $\rightarrow$ "Register-Memory"

Resposta: a)

④ adição em comp/para 2 $\leftrightarrow$ adição signed

$\rightarrow$ Quando a soma é signed, o bit do carry-out é ignorado, pois não tem significado

$\downarrow$
Logo, não é possível tirar nenhuma conclusão

Nota:

$\rightarrow$ Se fosse adição unsigned, o bit do carry-out simboliza a ocorrência de overflow

Resposta: c)

⑤ Codificação do JAL = codificação do J

opcode	offset
6 bits	26 bits

• $\$ra = PC + 4 = 0x0000A034 + 0x04 = \boxed{0x0000A038}$

• $PC = offset \ll 2 = 0x00000D4 \ll 2 = \boxed{0x00000350}$

$\downarrow$

$\ll 2$ $\begin{matrix} 0000 & 1101 & 0100 \\ \swarrow & \searrow & \searrow \\ 0011 & 0101 & 0000 \\ \underline{\quad} & \underline{\quad} & \underline{\quad} \\ 3 & 5 & 0 \end{matrix}$

Resposta: d)

⑥ Salto condicional $\rightarrow$ branches $\rightarrow$ "beq"

Logo, é endereçamento relativo ao PC com deslocamento

Resposta: c)

⑦ lui \$4, 0x36AC → \$4 = 0x36AC0000
 addi \$4, \$4, 0x6F80 → \$4 = 0x36AC6F80

sw \$4, 4(\$4) → Vai escrever o valor de \$4, no endereço = 0x36AC6F80 + 0x04

0x36AC6F84

0x36	0x-----87
0xAC	0x-----86
0x6F	0x-----85
0x80	0x36AC6F84 → end. memória

lb \$4, 6(\$4) → Vai escrever em \$4 o valor que está no endereço

0x36AC6F80 + 0x06 = 0x36AC6F86

valor de \$4 antes da execução da instrução lw

→ Olhando para o "desenho" da memória o valor que está no endereço 0x36AC6F86 é 0xAC

Como a instrução é "lb", temos de fazer a extensão de sinal

→ Logo, \$4 = 0xFFFFFFFFAC

Resposta: d)

⑧ Game para operações com sinal → $[-2^{n-1}, 2^{n-1}-1]$

Na instrução branch → offset tem 16 bits

BTA = (PC+4) + (offset << 2)

instrução seguinte

é o mesmo que acrescentar 2 zeros à direita do offset

Logo, offset "passa a ter" 18 bits

Por isso, $n = 18 \Rightarrow$ Logo, a gama = $[-2^{17}, 2^{17}-1]$

Resposta: a)

9) LO → fica com os 32 bits menos significativos da multiplicação

$$\$2 = 0xFFFFFFFFE = -2$$

$$\$3 = 0x00000003 = 3$$

$$\$2 * \$3 = -2 * 3 = -6 = \boxed{0xFFFFFFFFFA}$$

Resposta: d)

10) HI → fica com o resto da divisão

$$\$2 = 0xFFFFFFFFE = -2$$

$$\$3 = 0x00000003 = 3$$

$$\$2 / \$3 = -2 / 3$$

dividendo divisor

→ resto = -2, pois o resto fica sempre com o sinal do dividendo

↓ Logo

$$-2 = \boxed{0xFFFFFFFFE}$$

Resposta: b)

11) bge \$5, 0xA3, target → $(\$5 \geq 0xA3) = \sim(\$5 < 0xA3)$

$\boxed{\text{SlTi } \$1, \$5, 0xA3}$ → Se $\$5 < 0xA3$, $\$1 = 0x01$
Senão $\$1 = 0x00$

Logo, se $\$1 = 0x00$, então $\$5 \geq 0xA3$



Quando se verifica, é realizado o salto

Por isso, a segunda instrução é $\boxed{\text{bge } \$1, \$0, \text{target}}$

Resposta: a)

12) jr → não tem nenhuma restrição, logo pode saltar para qualquer endereço múltiplo de 4

Ou seja, de $\boxed{0x00000000 \text{ a } 0xFFFFFFFFC}$

↓
acaba em "C", porque tem de ser múltiplo de 4

Resposta: d)

13



- opcode

→ Qual o espaço de endereçamento para 130 instruções?

$$2^{10} = 1024$$

$$2^9 = 512$$

$$2^8 = 256$$

$$2^7 = 128$$

← Tem de ser 2^8 bytes

barramento de endereços

8 bits

Logo, o opcode tem 8 bits também

- offset

$$\rightarrow \text{Gamma} = [-32468, 3767] = \pm 32 \text{ K}$$

Logo, o offset tem 16 bits como
no MIPS

Report

Sendo que,



16 bits

$30 - 8 - 16 = 6$ bits $\rightarrow$ Logo ris tem 3 bits e rt
tambem tem 3 bits

→ Como r_s e r_t têm 3 bits, o número de registros internos é $2^3 = 8$ registros

Resposta: b)

14. O que se pretende

0000 0000 0000 0000 0000 0000 xxxx xxxx
negar

addi \$1, \$0, 0xFFFF → \$1 = 0xFFFF

nor \$3, \$3, \$1 $\rightarrow$ os 24 primeiros bits ficam a 0
os 8 últimos bits são negados

non

00	→	1
----	---	---

 $\odot \rightarrow \odot$

10 → 0

11 → 0

Resposta: d)

③

(15)

L1: .word 0, 1, 2 → reserva espaço para 3 words,
 ou seja, para 12 bytes
 L2: .ascii "NATAL" → reserva espaço do tamanho da
 string + o terminador '\0', sendo
 que cada caractere é 1 byte
 L3: .space 20 → o próximo endereço será
 múltiplo de 8 (2^3)
 → reserva espaço para 20 bytes

0x00000100	0x00	→ L1
01	0x00	
02	0x00	
03	0x00	
04	0x01	
05	0x00	
06	0x00	
07	0x00	
08	0x10	
09	0x00	
0A	0x00	
0B	0x00	
0C	'N'	→ L2
0D	'A'	
0E	'T'	
0F	'A'	
10	'L'	
11	'\0'	
12	?	→ align 3
13	?	
14	?	
15	?	
16	?	
17	?	
18	?	→ L3

Logo, L3 está no endereço 0x00000118

Resposta: a)

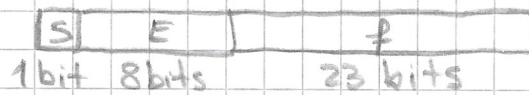
(16) int *p; → &p → aponta para o endereço
 *p → aponta para o conteúdo

Resposta: e)

(17) precisão dupla → excesso de 1023

Resposta: b)

18) menor quantidade positiva $\rightarrow 1,0000 \times 10^{-126}$



- $S = 0$, pois é um n° positivo
- $E = \text{expoente} + 127 = -126 + 127 = 1 = 0 \times 01$
- $f = 0 \times 000000$

Logo, $0 \mid 00000001 \mid 0000000000000000000000$
 $= 0 \times 00800000$

Resposta: d)

19) Resposta: a)

20) Resposta: c)

21) Resposta: a)

22) saída de ALUOut na 4ª fase

||
saída de ALUResult na 3ª fase

$\rightarrow$ Numa instrução "lw", na 3ª fase é calculado o endereço de memória

$$\text{end. mem} = (\text{conteúdo de rs}) + \text{offset}$$

Resposta: b)

23) A unidade de controlo do Pipeline não é uma máquina de estados, ou seja, é um circuito lógico combinatório

Os sinais de controlo são gerados na 2ª fase, ou seja, em ID

Resposta: d)

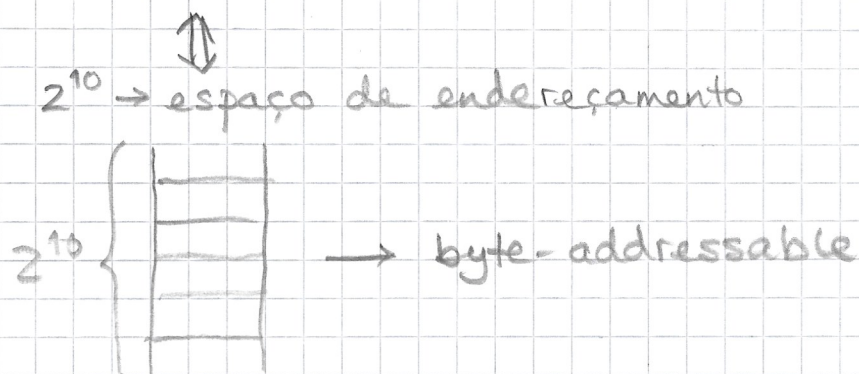
24) Resposta: b)

25) delayed-branch $\rightarrow$ Exemplo: $\text{beq } \$1, \$2, \text{end}$
 $\text{addi } \$3, \$3, 1$

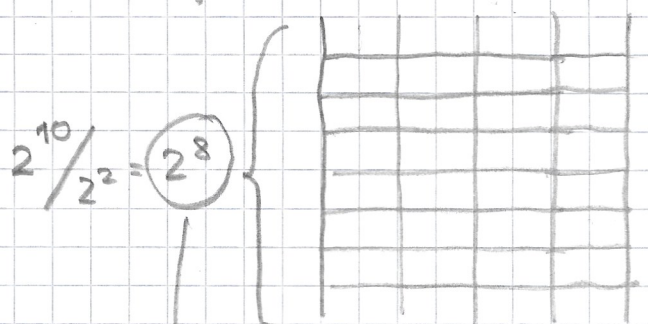
$\downarrow$
a instrução "addi" é realizada primeiro do que "beq", independentemente de saltar ou não

Resposta: a)

26) 1 KByte = 10 bits de barramento



$\rightarrow$ Se for word-addressable



espaço de endereçamento

$\downarrow$

Logo, o barramento é de 8 bits (A_7 a A_0)

• Como os endereços são sempre múltiplos de 4, os bits A_1 e A_0 são sempre 0 e, por isso, são descartados

$\downarrow$

8 bits — A_7 a A_2

Resposta: c)

27) frequência máxima de operação



tempo mais longo de todas as instruções $\rightarrow$ "lw"

$$t_{lw} = t_{RM} + \underbrace{\max(t_{RFR}, t_{uc}, t_{es})}_{\text{tempo mais longo desta 3 operações, pois são executadas ao mesmo tempo}} + t_{ALU} + t_{RM} + t_{RFW}$$

tempo mais longo desta
3 operações, pois são executadas
ao mesmo tempo

$$= 3 + \max(2, 1, 0) + 1 + 3 + 1$$

$$= 3 + 2 + 1 + 3 + 1 = 10 \text{ ns}$$

$$f = \frac{1}{t} = \frac{1}{10 \text{ ns}} = 100 \text{ MHz}$$

Resposta: b)

28) RegDst, JorD, MemtoReg são sinais de muxs, ou seja, só sabendo o valor deles, não podemos concluir qual a fase e a instrução que está a ser executada

Resposta: c)

29) LUI \$reg, imm

↓
escreve nos 16 bits mais significativos do \$reg, o valor de imm

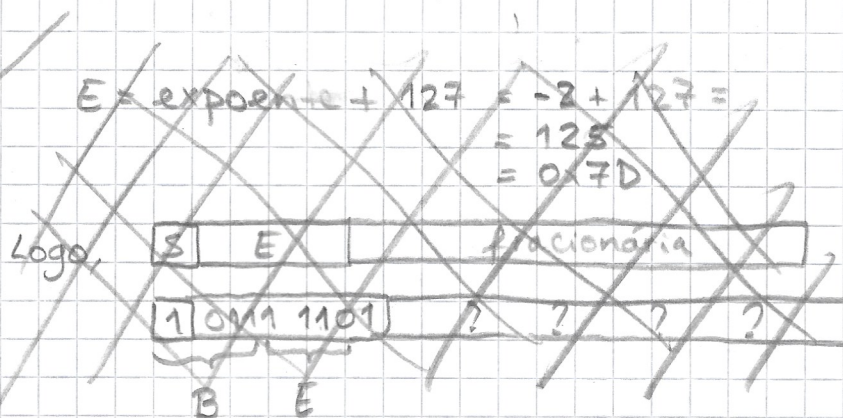
• Por isso, temos de acrescentar uma entrada no Mux3, pois é o mux que recebe o valor calculado na ALU numa instrução de "immediatos"

• E ligamos os 16 bits mais significativos aos 16 bits menos significativos do código máquina

Resposta: a)

30) Resposta: d)

31) $-40,625 \times 10^{-2} = -4,0625 \times 10^{-1} = -0,40625 \times 10^0$



• Como descobrir a parte fracionária em binário?

```

0,40625
x 2
0,81250
x 2
1,62500
0,62500
x 2
1,25000
0,25000
x 2
0,50000
x 2
1,00000
  
```

→ Parte fracionária = 0,01101₂

→ Parte inteira = 0

→ $0,40625_{10} = 0,01101_2 \times 10^0$
 $= 1,101_2 \times 10^{-2}$ ← normalizando

→ $E = \text{expoente} + 127 = -2 + 127 = 125_{10}$
 $= 0x7D_{16}$
 $= 0111 1101_2$

Logo fica, 1 0111 1101 1010000 - - - - -

= 0xBED00000

Resposta: b)

32) Resposta: d)

33) Estamos a trabalhar em base 10

↓ logo

$$n^{\circ} \text{ bits} = \log_2(10) = \frac{\log_{10}}{\log_2} = \frac{1}{0,3} = 3,3 \text{ bits}$$

→ Como queremos uma precisão de 4 casas decimais

$$n^{\circ} \text{ bits} = \lceil 4 \times 3,3 \rceil = \lceil 13,2 \rceil = 14 \text{ bits}$$

(arredonda para cima)

parte fracionária

• Sendo um formato de 20 bits,



Logo E ocupa 5 bits

→ Como queremos saber o excesso do expoente:

$$2^{n-1} - 1 = 2^{5-1} - 1 = 2^4 - 1 = 15$$

excesso de 15

Resposta: a)

34) addi
lw
add
and
* (beq
addi
and
sw

* como esta implementação tem o hard-branch, o "addi" é realizado antes do "beq", ou seja está dentro do ciclo

Resposta: c)

35) Fase WB de "add \$6, \$6, \$3" ↔ Fase ID de "beq \$3, \$2, L1"

Logo, o valor do registo PC é o valor de PC em "beq"

$$\begin{aligned} \hookrightarrow PC_{\text{beq}} &= PC_{\text{inst}} + 5 * (0x04) && \rightarrow (n^{\circ} \text{ de saltos}) \\ &= 0x00006024 + 0x00000014 \\ &= 0x00006038 \end{aligned}$$

Resposta: b)

36) No décimo ciclo, a instrução que está na Fase EX, ou seja, que está a usar a ALU é o "addi"

↓
Logo, o valor de saída da ALU é:

$$\$5 + 4 = 0x7A34 + 0x04$$

$$= 0x00007A38$$

Resposta: d)